Mathematical Booklet for the PINN / HJB / PMP Tutorial

Explaining the Mathematics Behind the Tutorial Notebooks

A guide to PINNs, HJB, PMP, nonlinear BVPs, PDEs, inverse problems, and nonlinear optimal control

This booklet follows the tutorial notebooks and focuses on the mathematics behind each solution. The goal is not to repeat code cell by cell, but to explain what problem is being solved, what the unknown is, how the equations are built, and why a particular method is the right one for that notebook.

Prepared by Mohammad Belal Irshaid

April 15, 2026

Preface

This booklet is written for upper-level undergraduate and early graduate students who want to understand the mathematics behind the tutorial notebooks. Throughout the tutorial we meet several different kinds of mathematical problems:

- solving an ODE or PDE,
- solving a nonlinear boundary-value problem,
- identifying an unknown physical parameter from sparse data,
- computing an optimal control input,
- learning either a value function or a trajectory with a neural network.

The central teaching message is simple: *the method should match the mathematics of the problem.* Some notebooks are naturally viewed as PINN residual-minimization problems. Others are naturally viewed through HJB or PMP. Some are bridges between classical control and neural methods.

How to Read This Booklet

We use a consistent visual legend throughout:

- Blue = state variables and solution fields, such as x, θ, ω, $y(x)$, $u(x,t)$.
- Red = control inputs or decision variables, such as $u(t)$.
- Green = objective, cost, loss, terminal cost, running cost.
- Orange = dynamics, PDEs, ODEs, residual equations, and constraints.
- Purple = value functions, Hamiltonians, Riccati terms, and costates.

Whenever a key concept first appears, it is written as a key term. After major formulas, a meaning box explains what the formula says in plain language. Translation boxes connect math notation to notebook implementation names.

Contents

1 Quick Background Chapter **1**

1.1 Core ideas before Notebook 01 1

1.2 A first map of the tutorial 2

2 Notebook 01 – Toy LQR HJB-PINN **3**

2.1 What problem is being solved? 3

2.2 Why this notebook matters 3

2.3 Mathematical model 3

2.4 Step-by-step derivation 4

2.5 How the neural network is used 5

2.6 How the loss or optimality equations are built 5

2.7 What the main plots/results mean 5

2.8 What students should remember 6

2.9 Common mistakes or misconceptions 6

3 Notebook 02 – PINN for the Nonlinear Bratu BVP **7**

3.1 What problem is being solved? 7

3.2 Why this notebook matters 7

3.3 Mathematical model 7

3.4 Step-by-step derivation 7

3.5 How the numerical method is used 8

3.6 What the main plots/results mean 8

3.7 What students should remember 9

3.8 Common mistakes or misconceptions 9

4 Notebook 03 – PINN for a Nonlinear Reaction-Diffusion PDE **10**

4.1 What problem is being solved? . . . 10
4.2 Why this notebook matters . . . 10
4.3 Mathematical model . . . 10
4.4 Step-by-step derivation . . . 10
4.5 How the neural network / numerical method is used . . . 11
4.6 How the loss is built . . . 11
4.7 What the main plots/results mean . . . 11
4.8 What students should remember . . . 12
4.9 Common mistakes or misconceptions . . . 12

5 Notebook 04 – Inverse PINN for Burgers Parameter Identification 13

5.1 What problem is being solved? . . . 13
5.2 Why this notebook matters . . . 13
5.3 Mathematical model . . . 13
5.4 Step-by-step derivation . . . 13
5.5 How the neural network / numerical method is used . . . 14
5.6 How identifiability should be understood . . . 14
5.7 What the main plots/results mean . . . 15
5.8 What students should remember . . . 15
5.9 Common mistakes or misconceptions . . . 15

6 Notebook 05 – Failure Case: Low-Viscosity Burgers 16

6.1 What problem is being solved? . . . 16
6.2 Why this notebook matters . . . 16
6.3 Mathematical model . . . 16
6.4 Step-by-step derivation . . . 16
6.5 How the neural network / numerical method is used . . . 17
6.6 What students should learn from the failure . . . 17
6.7 What the main plots/results mean . . . 17

6.8 What students should remember . . . 18
6.9 Common mistakes or misconceptions . . . 18

7 Notebook 06 – Scope Comparison: PINNs vs HJB/PMP 19

7.1 Main message . . . 19
7.2 Part A: Nonlinear optimal control for a pendulum . . . 19
7.3 Part B: Nonlinear Bratu BVP . . . 19
7.4 Side-by-side comparison . . . 20
7.5 Three categories to remember . . . 20

8 Notebook 07 – Nonlinear Optimal Control with a Trajectory PINN 21

8.1 What problem is being solved? . . . 21
8.2 Why this notebook matters . . . 21
8.3 Mathematical model . . . 21
8.4 Trajectory PINN formulation . . . 22
8.5 Approximate DP / HJB benchmark . . . 23
8.6 PMP benchmark . . . 24
8.7 Comparison chapter inside Notebook 07 . . . 24
8.8 What students should remember . . . 25
8.9 Common mistakes or misconceptions . . . 25

9 Connecting the Ideas Across All Notebooks 27

9.1 A synthesis of the tutorial . . . 27
9.2 ODE residuals vs PDE residuals . . . 27
9.3 Boundary, initial, and terminal conditions . . . 27
9.4 Direct learning vs inverse learning . . . 27
9.5 Value-function learning vs trajectory learning . . . 28
9.6 Feedback control vs open-loop control . . . 28
9.7 HJB vs PMP vs PINNs . . . 28
9.8 Decision tree for method choice . . . 29

9.9 Notation alignment table 29

10 Notation Glossary **30**

11 Final Big Picture Conclusion **31**

References **33**

Chapter 1 Quick Background Chapter

1.1 Core ideas before Notebook 01

An ordinary differential equation (ODE) describes how a quantity changes with respect to one independent variable, usually time. A partial differential equation (PDE) describes how a quantity depends on several independent variables, such as space and time. A boundary-value problem (BVP) asks for a function satisfying a differential equation together with conditions at the boundary of the domain. An initial-value problem (IVP) prescribes the state at the initial time and evolves forward.

An optimal control problem adds a new ingredient: among all admissible controls $u(t)$, we seek one that minimizes a cost J while respecting dynamics. The value function $V(x,t)$ tells us the best cost achievable starting from state x at time t. The Hamilton–Jacobi–Bellman (HJB) equation is the dynamic programming equation for this value function. The Pontryagin Maximum Principle (PMP) gives first-order necessary conditions using costates λ.

A physics-informed neural network (PINN) uses a neural network to represent an unknown function and then trains the network so that the governing equation, boundary conditions, and data constraints are approximately satisfied. Collocation points are the sample points where the residual is enforced. Residual minimization means we build a loss from the mismatch in the differential equation and train to make that mismatch small.

Concept	Mathematical object	What is unknown?	Typical method	Where it appears?
ODE	$\dot{x} = f(x,t)$	trajectory	time stepping, shooting, PMP	Notebooks 01, 06, 07
PDE	$u_t = \mathcal{N}[u]$	field $u(x,t)$	finite difference, finite element, PINN	Notebooks 03, 04, 05
BVP	$y'' = g(x,y)$ with endpoint conditions	function $y(x)$	shooting, collocation, `solve_bvp`, PINN	Notebooks 02, 06
Optimal control	dynamics + cost functional	control and often trajectory	HJB, PMP, direct methods, trajectory PINNs	Notebooks 01, 06, 07
Value function	$V(x,t)$	minimal future cost	HJB, dynamic programming, value PINNs	Notebooks 01, 07

Inverse problem	PDE + sparse observations	field and unknown parameter	PINNs, adjoint methods, Bayesian methods	Notebook 04
Residual minimization	$r = 0$ enforced approximately	network parameters	PINNs	Notebooks 01–05, 07

1.2 A first map of the tutorial

Before diving into notebook-specific details, keep the following distinctions in mind:

- In some chapters, u means a *control input* $u(t)$.
- In PDE chapters, u means a *solution field* $u(x, t)$.
- In HJB, the unknown is often a value function. In PMP, the unknowns are state trajectories, costates, and controls. In many PINN problems, the unknown is directly the solution function itself.

Chapter 2 Notebook 01 – Toy LQR HJB-PINN

2.1 What problem is being solved?

The notebook studies a scalar finite-horizon linear-quadratic regulator problem. The state is $x(t)$, the control is $u(t)$, and the dynamics are

$$\dot{x} = a\,x + b\,u, \qquad a = -0.5,\ b = 1.0.$$

The cost to be minimized is

$$J = \int_t^T \left(q\,x(s)^2 + r\,u(s)^2\right) ds + P_T\,x(T)^2,$$

with $q = 1.0$, $r = 0.1$, $T = 2.0$, and $P_T = 1.0$. The state domain used for training is approximately $x \in [-2, 2]$.

2.2 Why this notebook matters

This notebook is the cleanest bridge between classical optimal control and PINNs. The problem is simple enough that we know the analytic structure, yet rich enough to illustrate the HJB equation, the optimal control law, the Riccati ODE, and a neural approximation of the value function.

2.3 Mathematical model

The central unknown is the <u>finite-horizon value function</u>

$$V(x,t) = \min_{u(\cdot)} \left\{ \int_t^T \left(q\,x(s)^2 + r\,u(s)^2\right) ds + P_T\,x(T)^2 \right\}.$$

The value function tells us the best possible future cost if at time t we are currently at state $x = x$.

Intuition Before Derivation

For LQR, the system is linear and the cost is quadratic. That usually means the optimal cost itself has a quadratic shape in the state. So before doing any heavy derivation, we expect $V(x,t)$ to look like "some time-dependent coefficient" times x^2.

2.4 Step-by-step derivation

The HJB equation for this finite-horizon problem is

$$V_t + \min_u \Big(qx^2 + ru^2 + V_x(ax + bu)\Big) = 0, \qquad V(x,T) = P_T x^2.$$

Meaning of the Equation

At each point (x,t), HJB balances two things: the immediate running cost $qx^2 + ru^2$ and the future cost encoded by the gradient term $V_x(ax + bu)$. The optimal control is whichever u makes that bracket as small as possible.

To minimize the Hamiltonian term with respect to u, differentiate:

$$\frac{\partial}{\partial u}\Big(ru^2 + V_x(ax + bu)\Big) = 2ru + bV_x = 0.$$

Hence the optimal feedback law is

$$u^* = -\frac{b}{2r}V_x.$$

Now use the quadratic ansatz

$$V(x,t) = P(t)x^2.$$

Then

$$V_t = \dot{P}(t)x^2, \qquad V_x = 2P(t)x.$$

Substituting into the feedback law gives

$$u^*(t,x) = -\frac{b}{2r}(2P(t)x) = -\frac{b}{r}P(t)x.$$

Putting this into HJB and collecting the coefficient of x^2 yields the Riccati ODE:

$$\dot{P} = -\Big(2aP + q - \frac{b^2}{r}P^2\Big), \qquad P(T) = P_T.$$

Meaning of the Equation

The value function is not arbitrary. Once we assume it is quadratic, the entire PDE collapses to a one-dimensional ODE for $P(t)$. This is why LQR is so important pedagogically: it turns a control PDE into a tractable benchmark.

2.5 How the neural network is used

Instead of forcing the quadratic form, the PINN learns a neural approximation $V_\theta(x,t)$. Automatic differentiation computes

$$V_t, \qquad V_x.$$

Then the notebook defines the PINN feedback law by the same minimizing formula:

$$u^*(x,t) = -\frac{b}{2r}V_x.$$

2.6 How the loss or optimality equations are built

The HJB residual is

$$R(x,t) = V_t + qx^2 + r(u^*)^2 + V_x(ax + bu^*).$$

At the terminal time we must match the terminal cost:

$$V(x,T) = P_T x^2.$$

Therefore the training loss is

$$\mathcal{L} = \text{mean}(R^2) + \lambda_{\text{term}}\ \text{mean}\left(V(x,T) - P_T x^2\right)^2.$$

Translation to Notebook Implementation

In the notebook, the network is `ValueNet`. Interior space-time points are sampled in $x \in [-2,2]$, $t \in [0,2]$. Terminal points are sampled at $t = T$. The code uses `hjb_residual(V, x, t, cfg_lqr)` to build the physics term and adds a terminal penalty with weight `lambda_term`.

2.7 What the main plots/results mean

The analytic Riccati solution provides a benchmark because for this notebook we know the exact structure of the optimal solution. The notebook estimates a learned coefficient $\widehat{P}(t)$ by fitting

$$V_\theta(x,t) \approx \widehat{P}(t)x^2$$

in a least-squares sense on each time slice. Comparing $\widehat{P}(t)$ with the Riccati solution $P(t)$ is meaningful because, if the PINN has learned the right value function, its dependence on x should be close to quadratic.

The value-surface plot shows the geometry of $V(x,t)$ over the state-time domain. The residual heatmap shows where the HJB equation is being satisfied well or poorly. The closed-loop trajectory comparison checks whether the PINN-induced feedback behaves like the analytic optimal feedback.

Bridge Idea

Why this notebook is the bridge from classical optimal control to PINNs. Classical control gives an exact derivation: HJB $\rightarrow$ minimizing feedback $\rightarrow$ Riccati ODE. The PINN keeps the HJB logic but replaces the hand-derived form by a trainable function V_θ. Students see both the old idea and the new computational tool in one place.

2.8 What students should remember

- The unknown is the value function, not directly the state trajectory.
- HJB gives a PDE for the optimal cost-to-go.
- The minimizing control comes from differentiating the Hamiltonian expression with respect to u.
- For scalar LQR, the quadratic ansatz reduces HJB to a Riccati ODE.
- A value-function PINN can be checked against the analytic Riccati benchmark.

2.9 Common mistakes or misconceptions

Common Confusion

Do not confuse the terminal condition $V(x,T) = P_T x^2$ with an initial condition. HJB for a finite-horizon problem runs *backward in time*. Also, the symbol u here means a control input; later in PDE notebooks, the symbol u will mean a field.

Check Your Understanding

- What is the unknown in this notebook: a trajectory, a control, or a value function?
- Why is the Riccati ODE a valid benchmark here?
- What does the HJB residual measure?
- Why does the terminal condition enter the loss explicitly?

Chapter 3 Notebook 02 – PINN for the Nonlinear Bratu BVP

3.1 What problem is being solved?

The Bratu problem is

$$y''(x) + \lambda e^{y(x)} = 0, \qquad x \in [0, 1],$$

with boundary conditions

$$y(0) = 0, \qquad y(1) = 0, \qquad \lambda = 1.0.$$

3.2 Why this notebook matters

This notebook shows a canonical PINN use case: solving a nonlinear boundary-value problem directly from the differential equation and boundary conditions.

3.3 Mathematical model

The unknown is the function $y(x)$. There is **no control input**, **no cost functional**, and **no optimization over policies**. So HJB and PMP are not the natural frameworks here.

Intuition Before Derivation

Think of Bratu as a "find the curve" problem, not a "choose the best action" problem. We are not deciding anything over time. We simply want a function whose curvature and nonlinear source term balance each other while satisfying endpoint conditions.

3.4 Step-by-step derivation

Let the PINN output be $y_\theta(x)$. By automatic differentiation, we compute

$$y'_\theta(x), \qquad y''_\theta(x).$$

The residual is then

$$r(x) = y''_\theta(x) + \lambda e^{y_\theta(x)}.$$

Boundary conditions are enforced with a boundary loss:

$$\mathcal{L}_{\text{bc}} = |y_\theta(0)|^2 + |y_\theta(1)|^2.$$

The PDE/BVP residual loss is

$$\mathcal{L}_{\text{pde}} = \text{mean}_{x_f}\,(r(x_f)^2).$$

The total loss is a weighted combination:

$$\mathcal{L} = \mathcal{L}_{\text{pde}} + w_{\text{bc}}\mathcal{L}_{\text{bc}}.$$

Meaning of the Equation

At interior collocation points, the network is asked to satisfy the differential equation. At the endpoints, it is asked to satisfy the two Dirichlet conditions. Training searches for one function that does both at once.

3.5 How the numerical method is used

The notebook uses a tanh multilayer perceptron for y_θ, samples interior collocation points in $[0, 1]$, and uses exactly two boundary points $x = 0$ and $x = 1$. A classical reference solution is computed using `scipy.solve_bvp`. That reference is not needed to train the PINN, but it is extremely useful for validation.

Translation to Notebook Implementation

The notebook defines `BratuPINN`, computes derivatives with `bratu_derivatives(model, x)`, and trains with `N_COLLOCATION = 128`, `EPOCHS = 4000`, and a tanh network of width 64 and depth 3.

3.6 What the main plots/results mean

The learned solution vs. the `solve_bvp` reference tells us whether the PINN found the correct nonlinear profile. The absolute error curve shows where the approximation is most difficult. The residual curve shows how well the learned function satisfies the equation pointwise. The training-loss plot shows whether the PDE and boundary penalties are being reduced consistently.

Bridge Idea

Why a PINN is natural for BVPs. The equation itself tells us what valid solutions look like, and the boundary conditions tell us which member of that family we want. PINNs turn both pieces directly into a loss, so they fit the structure of a BVP very naturally.

3.7 What students should remember

- The unknown is a function $y(x)$, not a control law.
- There is no HJB, no PMP, and no costate here.
- PINNs use autograd to compute derivatives inside the residual.
- Boundary conditions are separate constraints and must appear in the loss.

3.8 Common mistakes or misconceptions

Common Confusion

Students often see a training loss and assume the problem must be an optimization problem in the control sense. Here the optimization is only a computational device for fitting a function. The underlying mathematical problem is still a nonlinear BVP.

Check Your Understanding

- What is the unknown in Bratu?
- Why are HJB and PMP not natural here?
- What does the residual $r(x)$ measure?
- Why are the endpoint conditions added separately to the loss?

Chapter 4 Notebook 03 – PINN for a Nonlinear Reaction-Diffusion PDE

4.1 What problem is being solved?

The notebook studies the Fisher–KPP equation

$$u_t = Du_{xx} + Ru(1-u), \qquad x \in [0,1],\ t \in [0,T],$$

with $D = 0.01$, $R = 2.0$, $T = 0.5$, initial condition

$$u(x,0) = 0.35 \exp\big(-60(x-0.35)^2\big),$$

and boundary conditions

$$u(0,t) = 0, \qquad u(1,t) = 0.$$

4.2 Why this notebook matters

This notebook is the first full space-time PDE chapter. It teaches students that the unknown can be a field $u(x,t)$, not merely a number-valued trajectory.

4.3 Mathematical model

The diffusion term Du_{xx} smooths the solution in space. The reaction term $Ru(1-u)$ models local growth with saturation. Since both space and time appear, the unknown is a field over a two-dimensional domain.

Intuition Before Derivation

Diffusion tries to flatten spatial variation. Reaction tries to amplify or damp the field locally. The solution is the outcome of these two effects acting together over time.

4.4 Step-by-step derivation

Let the neural network approximate the field by $u_\theta(x,t)$. Automatic differentiation gives

$$u_t, \qquad u_x, \qquad u_{xx}.$$

Move everything to one side to define the residual:

$$r(x,t) = u_t - Du_{xx} - Ru(1-u).$$

Then define

$$\begin{aligned}
\mathcal{L}_{\text{pde}} &= \text{mean}_{(x_f,t_f)}\left(r(x_f,t_f)^2\right),\\
\mathcal{L}_{\text{bc}} &= \text{mean}_{t_b}\left(u_\theta(0,t_b)^2 + u_\theta(1,t_b)^2\right),\\
\mathcal{L}_{\text{ic}} &= \text{mean}_{x_0}\left(u_\theta(x_0,0) - u_0(x_0)\right)^2.
\end{aligned}$$

The total loss is a weighted sum:

$$\mathcal{L} = \mathcal{L}_{\text{pde}} + w_{\text{bc}}\mathcal{L}_{\text{bc}} + w_{\text{ic}}\mathcal{L}_{\text{ic}}.$$

4.5 How the neural network / numerical method is used

The notebook uses many interior space-time collocation points, random samples on the spatial boundaries, random samples on the initial line $t = 0$, and a tanh MLP. A finite-difference solver is used to generate a reference solution for comparison.

Translation to Notebook Implementation

The implementation uses `FisherPINN`, 4000 interior collocation points per epoch, 300 random boundary time samples, and 300 random initial-condition samples. Training runs for 3500 epochs with loss

$$\mathcal{L} = \mathcal{L}_{\text{pde}} + 10\mathcal{L}_{\text{bc}} + 10\mathcal{L}_{\text{ic}}.$$

4.6 How the loss is built

Notice the important distinction in notation: here $u(x,t)$ is a *PDE field*, not a control input. That is why this notebook is not an optimal control problem. There is no minimizing over actions. Instead, the loss measures whether the candidate field satisfies the PDE and the side conditions.

4.7 What the main plots/results mean

The reference heatmap shows the evolution of the reaction-diffusion field over space and time. The PINN prediction heatmap should visually resemble it. The absolute error heatmap shows where mismatch concentrates. Time-slice comparisons provide one-dimensional cross-sections that are easier to inspect. The residual heatmap highlights where the PDE is least well satisfied. Loss curves tell whether the PDE, boundary, and initial constraints are being balanced during training.

Bridge Idea

Why the unknown here is a field, not a control policy. At every point (x, t) we seek the value of the physical state u. We are not choosing an action to influence the system; we are reconstructing the field that satisfies a PDE.

4.8 What students should remember

- This is a PDE problem in space and time.
- Diffusion and reaction play different physical and mathematical roles.
- The residual is formed by moving all PDE terms to one side.
- Initial and boundary conditions are distinct and both matter.

4.9 Common mistakes or misconceptions

Common Confusion

Do not confuse the initial condition with a boundary condition. The initial condition lives on the time edge $t = 0$, while boundary conditions live on the spatial edges $x = 0$ and $x = 1$. A space-time PDE needs both kinds of information.

Check Your Understanding

- What is the unknown in this notebook?
- What do diffusion and reaction each do?
- Why is this not an optimal control problem?
- What is being sampled at collocation points?

Chapter 5 Notebook 04 – Inverse PINN for Burgers Parameter Identification

5.1 What problem is being solved?

The notebook studies the viscous Burgers equation

$$u_t + uu_x = \nu u_{xx}, \qquad x \in [-1, 1],\ t \in [0, 1],$$

with initial condition

$$u(x, 0) = -\sin(\pi x),$$

and unknown viscosity ν. The true value used to generate synthetic reference data is $\nu_{\text{true}} = 0.05$. The notebook observes only 300 random sparse samples from the reference field.

5.2 Why this notebook matters

This chapter introduces an inverse problem. We are no longer solving only for a field; we are learning both the field $u(x, t)$ and an unknown physical parameter ν.

5.3 Mathematical model

The direct problem would be: given ν, solve the PDE. The inverse problem is harder: infer ν from sparse observations while also reconstructing the field.

Intuition Before Derivation

Sparse data alone may not tell us the viscosity reliably, because many imperfect fields might pass near a small set of points. The PDE acts like a physics-based regularizer: it rules out fields that fit the data but violate the governing law.

5.4 Step-by-step derivation

Let the network output be $u_\theta(x, t)$, and let ν be a trainable scalar parameter. The notebook enforces positivity by introducing a raw parameter and mapping it through a softplus-type function:

$$\nu = \log(1 + e^\eta) + 10^{-5}.$$

The residual becomes

$$r(x,t) = u_t + uu_x - \nu u_{xx}.$$

The loss has several pieces:

$$\begin{aligned}
\mathcal{L}_{\text{physics}} &= \text{mean}(r^2), \\
\mathcal{L}_{\text{data}} &= \text{mean}\left(u_\theta(x_i,t_i) - u_i^{\text{obs}}\right)^2, \\
\mathcal{L}_{\text{bc}} &= \text{mean}\left(u_\theta(-1,t_b)^2 + u_\theta(1,t_b)^2\right), \\
\mathcal{L}_{\text{ic}} &= \text{mean}\left(u_\theta(x_0,0) + \sin(\pi x_0)\right)^2.
\end{aligned}$$

The notebook combines boundary and initial losses and forms

$$\mathcal{L} = \mathcal{L}_{\text{physics}} + 20\mathcal{L}_{\text{data}} + 10\mathcal{L}_{\text{bc/ic}}.$$

Meaning of the Equation

The data term says "stay close to observed measurements." The physics term says "obey Burgers everywhere." The BC/IC term says "match the known side conditions." Learning ν works because changing ν changes which fields satisfy the PDE.

5.5 How the neural network / numerical method is used

The notebook uses a tanh MLP for the field, learns a separate trainable viscosity parameter, samples 5000 interior collocation points per epoch, and uses 400 random samples each for the boundary and initial terms. The reference field is generated numerically using a classical time integrator.

5.6 How identifiability should be understood

Identifiability asks whether the unknown parameter can in principle be inferred from the available information. Here sparse observations alone may be too weak, especially if they miss regions where viscosity matters strongly. Physics helps because only some (u,ν) pairs can satisfy Burgers consistently over the domain. The PINN therefore performs a joint fit of the field and the parameter.

Translation to Notebook Implementation

The notebook class is `InverseBurgersPINN`. The parameter is stored as `raw_nu` and converted by `softplus`. The training history records `physics`, `data`, `bc_ic`, and the learned `nu` over epochs.

5.7 What the main plots/results mean

The reference Burgers solution is the target field. The PINN prediction shows the reconstructed field. The absolute error with sparse observations overlaid is pedagogically useful: students can see both where data are available and how physics fills in the gaps. The learned-ν-versus-epoch plot reveals whether parameter identification converges toward ν_{true}. Loss curves show the trade-off between physics and data fitting.

Bridge Idea

What makes inverse PINNs powerful. They can combine sparse measurements with governing equations to estimate hidden quantities that are not directly observed. This is one of the strongest conceptual use cases for PINNs.

5.8 What students should remember

- This is an inverse problem, not just a forward PDE solve.
- The unknown includes both a field and a parameter.
- Physics regularization helps make sparse-data inference possible.
- Positivity constraints on physical parameters can be built into the parameterization.

5.9 Common mistakes or misconceptions

Common Confusion

Do not assume that more trainable quantities automatically mean better learning. Jointly learning u and ν is only meaningful because the PDE tightly couples them. Without that structure, parameter estimates can drift or become non-identifiable.

Check Your Understanding

- What makes this an inverse problem?
- Why is sparse data alone often not enough?
- What role does the physics residual play in identifying ν?
- Why enforce positivity on ν?

Chapter 6 Notebook 05 – Failure Case: Low-Viscosity Burgers

6.1 What problem is being solved?

The notebook revisits Burgers,

$$u_t + uu_x = \nu u_{xx}, \qquad x \in [-1, 1],\ t \in [0, 1],$$

but now with low viscosity $\nu = 0.003$ and initial condition $u(x, 0) = -\sin(\pi x)$.

6.2 Why this notebook matters

This chapter is intentionally pedagogical because it shows a genuine limitation of standard PINNs. It is not a sabotage example. The setup is fair, but the regime is hard.

6.3 Mathematical model

Small viscosity means weak diffusion. Weak diffusion allows steep fronts to develop. Steep fronts are hard for smooth tanh networks because they tend to learn low-frequency, smooth structure first.

Intuition Before Derivation

If the true solution has a sharp transition layer, a smooth MLP may blur that layer even when the average residual is not terrible. This is one way to understand spectral bias: low-frequency content is easier to learn than localized sharp structure.

6.4 Step-by-step derivation

The field network still outputs $u_\theta(x, t)$, and the residual is still

$$r(x, t) = u_t + uu_x - \nu u_{xx}.$$

Boundary and initial conditions are imposed similarly to Notebook 04:

$$\begin{aligned} \mathcal{L}_{\text{physics}} &= \text{mean}(r^2), \\ \mathcal{L}_{\text{bc/ic}} &= \mathcal{L}_{\text{bc}} + \mathcal{L}_{\text{ic}}, \\ \mathcal{L} &= \mathcal{L}_{\text{physics}} + 10\mathcal{L}_{\text{bc/ic}}. \end{aligned}$$

6.5 How the neural network / numerical method is used

The notebook uses the same general style of tanh MLP field approximator as the inverse Burgers notebook, but here ν is fixed rather than learned. Training uses 5000 interior collocation points and 400 random samples each for boundary and initial losses.

6.6 What students should learn from the failure

This notebook teaches several important lessons:

- PINNs are not automatically reliable in every regime.
- A small average residual does not guarantee sharp features are captured accurately.
- Smooth approximators may smear steep gradients.
- Hard PDE regimes may require adaptive sampling, domain decomposition, Fourier features, shock-aware formulations, or other enhanced architectures.

6.7 What the main plots/results mean

The reference low-viscosity solution shows steep structures emerging. The vanilla PINN prediction often looks smoother. The absolute error heatmap shows where that mismatch is concentrated. Slice plots through the sharp-front region make the failure especially clear. The residual heatmap may still look deceptively acceptable in some regions, which is an important teaching point.

Bridge Idea

Failure is informative: what this notebook teaches us. A method's weaknesses matter just as much as its strengths. This chapter trains students to inspect what a model gets wrong, not just whether the loss goes down.

6.8 What students should remember

- Low viscosity can create multiscale structure.
- Standard tanh PINNs may under-resolve steep fronts.
- Residual minimization and visual accuracy are related but not identical.

6.9 Common mistakes or misconceptions

Common Confusion

It is a mistake to conclude that "PINNs do not work." The right conclusion is narrower: *this standard PINN struggles in this difficult regime*. Matching the method to the regime still matters.

Check Your Understanding

- Why does low viscosity make the problem harder?
- What is spectral bias in plain language?
- Why can a residual loss be small while sharp fronts are still inaccurate?
- What kinds of enhancements might help?

Chapter 7 Notebook 06 – Scope Comparison: PINNs vs HJB/PMP

7.1 Main message

Method choice should follow *problem structure*, not fashion. This notebook does not argue that one method always wins. Instead, it asks: what is the mathematical object we are trying to compute?

7.2 Part A: Nonlinear optimal control for a pendulum

The pendulum setup uses horizon $T = 2.0$, damping 0.2, control penalty $r_u = 0.1$, initial state $q(0) = 1.0$, $\omega(0) = 0.0$, and terminal weights $q_f = 8.0$, $w_f = 1.0$. The dynamics are of the form

$$\dot{q} = \omega, \qquad \dot{\omega} = -\sin q - 0.2\,\omega + u.$$

The Hamiltonian idea is

$$H = L(q, \omega, u) + \lambda_q \omega + \lambda_\omega(-\sin q - 0.2\omega + u).$$

Stationarity with respect to the control gives

$$\frac{\partial H}{\partial u} = 2r_u u + \lambda_\omega = 0 \quad \Rightarrow \quad u = -\frac{\lambda_\omega}{2r_u}.$$

This is exactly the kind of structure where PMP or HJB is natural, because the task is to choose a control that minimizes a cost subject to dynamics.

7.3 Part B: Nonlinear Bratu BVP

Now compare with Bratu:

$$y'' + \lambda e^y = 0, \qquad y(0) = y(1) = 0.$$

Here the unknown is simply a state function satisfying an equation and boundary conditions. There is no control law being optimized. So the natural abstraction is direct equation solving, and a PINN fits that structure cleanly.

7.4 Side-by-side comparison

Problem class	Unknown	Mathematical target	Natural method	Strengths	Limitations
Nonlinear optimal control	control, trajectory, maybe value function	minimize cost subject to dynamics	HJB / PMP	directly encodes optimality	can be expensive or sensitive
Nonlinear BVP		satisfy ODE + boundary conditions	PINN / collocation / `solve_bvp`	simple direct formulation	training can be delicate
Inverse parameter ID	field + parameter	fit data and physics simultaneously	inverse PINN	uses sparse data + structure	identifiability may be weak

7.5 Three categories to remember

- **Nonlinear optimal control:** unknowns include choices or policies; PMP and HJB are mathematically natural.
- **Nonlinear BVP/PDE solving:** unknown is the solution function or field itself; PINNs are natural residual solvers.
- **Inverse identification:** unknown includes hidden parameters; PINNs are useful because they fuse data and governing laws.

Bridge Idea

Choose the method that matches the math. If there is a cost-minimizing control, think HJB or PMP. If there is only a differential equation with side conditions, think direct PDE/BVP solving. If hidden parameters must be inferred, think inverse methods and identifiability.

Check Your Understanding

- What is the unknown in the pendulum example?
- What is the unknown in Bratu?
- Why does "training a network" not automatically mean the underlying math is the same?

Chapter 8 Notebook 07 – Nonlinear Optimal Control with a Trajectory PINN

8.1 What problem is being solved?

This notebook studies nonlinear pendulum optimal control with state

$$x(t) = \begin{bmatrix} \theta(t) \\ \omega(t) \end{bmatrix},$$

and dynamics

$$\dot{\theta} = \omega, \qquad \dot{\omega} = -\sin(\theta) - c\omega + u, \qquad c = 0.1.$$

The goal is to move the pendulum toward the reference $(\theta_{\text{ref}}, \omega_{\text{ref}}) = (0, 0)$ from the initial condition $(\theta(0), \omega(0)) = (1.0, 0.0)$ over horizon $T = 3.0$, with bounded control $|u| \leq 2.0$.

8.2 Why this notebook matters

This chapter is one of the most important in the tutorial because it compares three viewpoints on the same nonlinear control problem:

- a **trajectory PINN**,
- an approximate **dynamic programming / HJB** benchmark,
- a **PMP** benchmark.

Students can see what each framework solves and what it returns.

8.3 Mathematical model

The running cost is

$$L(x, u) = q_\theta(\theta - \theta_{\text{ref}})^2 + q_\omega \omega^2 + r\, u^2 + \alpha(\theta - \theta_{\text{ref}})^4,$$

with parameters

$$q_\theta = 1.0, \quad q_\omega = 0.1, \quad r = 0.01, \quad \alpha = 0.5.$$

The terminal cost is

$$\Phi(x(T)) = q_T(\theta(T) - \theta_{\text{ref}})^2 + q_{\omega,T}\omega(T)^2,$$

with $q_T = 5.0$ and $q_{\omega,T} = 1.0$.

The total objective is

$$J = \int_0^T L(x(t), u(t))\, dt + \Phi(x(T)).$$

Intuition Before Derivation

This problem is nonlinear in two ways. First, the dynamics include $\sin(\theta)$. Second, the cost includes a quartic term $(\theta - \theta_{\text{ref}})^4$. So we should not expect closed-form LQR-style formulas.

8.4 Trajectory PINN formulation

This notebook does **not** learn a value function. Instead it learns three time-dependent outputs directly:

$$\theta_\phi(t), \qquad \omega_\phi(t), \qquad u_\psi(t).$$

This is a crucial distinction from Notebook 01.

The dynamics residuals are

$$\begin{aligned} r_\theta(t) &= \dot{\theta}_\phi(t) - \omega_\phi(t), \\ r_\omega(t) &= \dot{\omega}_\phi(t) - \big(-\sin(\theta_\phi(t)) - c\omega_\phi(t) + u_\psi(t)\big). \end{aligned}$$

Initial-condition losses enforce

$$\theta_\phi(0) = 1.0, \qquad \omega_\phi(0) = 0.0.$$

The control is bounded smoothly by

$$u(t) = u_{\max} \tanh(u_{\text{raw}}(t)), \qquad u_{\max} = 2.0.$$

Meaning of the Equation

The hyperbolic tangent keeps the control inside the admissible interval automatically. This avoids a hard projection during PINN training while preserving differentiability.

The running cost is approximated on a time grid using trapezoidal quadrature, so

$$\mathcal{L}_{\text{cost}} \approx \text{Trapz}\left(L(\theta_\phi(t_i), \omega_\phi(t_i), u_\psi(t_i))\right) + \Phi(x(T)).$$

The dynamics and initial-condition losses are

$$\begin{aligned} \mathcal{L}_{\text{dyn}} &= \text{mean}(r_\theta^2) + \text{mean}(r_\omega^2), \\ \mathcal{L}_{\text{ic}} &= |\theta_\phi(0) - 1|^2 + |\omega_\phi(0)|^2. \end{aligned}$$

The total training objective is

$$\mathcal{L} = \lambda_{\text{dyn}}\mathcal{L}_{\text{dyn}} + \lambda_{\text{ic}}\mathcal{L}_{\text{ic}} + \lambda_{\text{cost}}\mathcal{L}_{\text{cost}},$$

with notebook weights

$$\lambda_{\text{dyn}} = 50, \qquad \lambda_{\text{ic}} = 50, \qquad \lambda_{\text{cost}} = 1.$$

Translation to Notebook Implementation

The notebook uses a `TrajectoryPINN` with width 64, depth 3, 201 time points, learning rate 2×10^{-3}, and 2500 epochs. The state and control are learned as continuous-time functions of t, and autograd differentiates them with respect to time.

8.5 Approximate DP / HJB benchmark

The benchmark discretizes the state space and time:

- θ-grid on $[-\pi, \pi]$,
- ω-grid on $[-3, 3]$,
- about 61 points in each state dimension,
- about 61 time points,
- coarse control set $\{-2, -1, 0, 1, 2\}$.

Backward dynamic programming applies the Bellman recursion

$$V_k(x) = \min_{u \in \mathcal{U}} \Big(\Delta t \, L(x, u) + V_{k+1}(x^+) \Big),$$

where x^+ is the next state predicted over one time step. Interpolation is used between value-grid slices.

Meaning of the Equation

This benchmark gives an approximate value-function viewpoint. It is not a perfect continuous HJB solve, but it captures the dynamic-programming logic and produces an approximate feedback policy.

The curse of dimensionality also becomes visible here: even for a two-dimensional state, a dense state-time grid already costs noticeable computation.

8.6 PMP benchmark

For the same problem, define the Hamiltonian

$$H(x, u, \lambda) = L(x, u) + \lambda_1 \omega + \lambda_2 \big(- \sin\theta - c\omega + u \big).$$

Stationarity yields

$$\frac{\partial H}{\partial u} = 2ru + \lambda_2 = 0 \quad \Rightarrow \quad u^* = -\frac{\lambda_2}{2r}.$$

The terminal conditions for costates come from the terminal cost:

$$\lambda_1(T) = 2q_T(\theta(T) - \theta_{\text{ref}}), \qquad \lambda_2(T) = 2q_{\omega,T}(\omega(T) - \omega_{\text{ref}}).$$

The costate equations are obtained from

$$\dot{\lambda} = -\frac{\partial H}{\partial x},$$

which gives

$$\begin{aligned} \dot{\lambda}_1 &= -\Big(2q_\theta(\theta - \theta_{\text{ref}}) + 4\alpha(\theta - \theta_{\text{ref}})^3 - \lambda_2 \cos\theta\Big), \\ \dot{\lambda}_2 &= -\Big(2q_\omega \omega + \lambda_1 - c\lambda_2\Big). \end{aligned}$$

The notebook uses a forward-backward sweep with a relaxed control update and then clips controls into $[-u_{\max}, u_{\max}]$.

Bridge Idea

Why PMP is elegant but sensitive. PMP turns the control problem into a coupled boundary-value problem in state and costate variables. This is mathematically beautiful, but numerically sensitive because the forward and backward pieces must agree.

8.7 Comparison chapter inside Notebook 07

The trajectory plots compare how the three methods steer the pendulum. The control plots compare the time profiles of the computed controls. The phase portrait compares state-space geometry. The objective summary table compares the achieved cost values.

Method	What it solves	What it outputs	Major strengths	Major weaknesses

Trajectory PINN	direct optimal trajectory fitting with residual penalties	open-loop trajectory and control	flexible, differentiable, direct continuous-time representation	no value function, training-weight tuning required
DP/HJB benchmark	approximate Bellman recursion on a grid	approximate value and feedback policy	conceptually close to feedback control	curse of dimensionality, coarse discretization
PMP	first-order optimality conditions	state, costate, and control trajectories	mathematically sharp and interpretable	sensitive forward-backward coupling

Bridge Idea

Trajectory PINN vs Value-function PINN: what is the difference? A value-function PINN learns $V(x,t)$, from which a feedback law may be derived. A trajectory PINN learns a particular optimal trajectory and control over one horizon. The first is closer to dynamic programming; the second is closer to direct trajectory optimization.

8.8 What students should remember

- This notebook is a nonlinear optimal control problem.
- The trajectory PINN learns $\theta(t)$, $\omega(t)$, and $u(t)$ directly.
- The DP benchmark gives an approximate value-function viewpoint.
- PMP gives necessary conditions through costates and stationarity.
- Different methods answer different questions even on the same physical system.

8.9 Common mistakes or misconceptions

Common Confusion

Do not say "the PINN solves HJB" for this notebook. It does not. The trajectory PINN solves a direct residual-penalized trajectory optimization problem. HJB appears only in the separate dynamic-programming benchmark.

Check Your Understanding

- What is the unknown in the trajectory PINN?
- How is this different from Notebook 01?
- Why is DP naturally associated with feedback control?
- Why can PMP be numerically sensitive?

Chapter 9 Connecting the Ideas Across All Notebooks

9.1 A synthesis of the tutorial

Across the notebooks, the same computational pattern appears repeatedly: define an unknown function, derive governing equations, build residuals, impose side conditions, and compare against a benchmark when possible. But the *meaning* of the unknown changes from chapter to chapter.

9.2 ODE residuals vs PDE residuals

An ODE residual usually depends on one independent variable, for example

$$r(t) = \dot{x}(t) - f(x(t), u(t), t).$$

A PDE residual depends on several independent variables, for example

$$r(x, t) = u_t - \mathcal{N}[u].$$

The PINN machinery is similar in both cases, but the mathematical object being learned can be very different.

9.3 Boundary, initial, and terminal conditions

- **Boundary conditions** constrain a solution on the spatial boundary, as in Bratu or Fisher–KPP.
- **Initial conditions** specify the state at the starting time, as in reaction-diffusion and Burgers.
- **Terminal conditions** constrain the endpoint cost or value, as in HJB and PMP formulations.

Students should always ask: *where in the domain is the condition applied?*

9.4 Direct learning vs inverse learning

Notebook 02 and Notebook 03 are direct problems: the unknown is the solution function or field. Notebook 04 is an inverse problem: the unknown includes a hidden parameter. Inverse problems are usually harder because several explanations may fit limited data unless physics reduces the ambiguity.

9.5 Value-function learning vs trajectory learning

Notebook 01 learns a value function. Notebook 07 learns a trajectory and a control signal. This is a major conceptual difference:

- a value function supports **feedback** thinking,
- a trajectory method usually produces an **open-loop** plan for one scenario.

9.6 Feedback control vs open-loop control

If the control is written as $u^*(x, t)$, it is state-dependent feedback. If the control is learned as $u(t)$ only, it is open-loop over the prescribed horizon. HJB naturally leads toward feedback. Direct trajectory PINNs often produce open-loop controls unless additional structure is added.

9.7 HJB vs PMP vs PINNs

Question	HJB	PMP	PINNs	Best match in tutorial
What is the best future cost?	yes, via value function	indirectly	sometimes, if value PINN is used	Notebooks 01, 07 benchmark
What optimality conditions hold?	globally via dynamic programming	locally via costates	only if built into loss	Notebooks 01, 06, 07
How do we solve a BVP/PDE directly?	not natural	not natural	very natural	Notebooks 02, 03
Can we identify hidden parameters from sparse data?	not typical	not typical	very natural	Notebook 04
What happens in hard multiscale regimes?	theory depends on formulation	sensitivity possible	may struggle badly	Notebook 05

9.8 Decision tree for method choice

Decision Tree

1. Is there a cost functional and a control input to optimize?
   - If yes, think HJB, PMP, dynamic programming, or direct optimal control methods.
   - If no, go to step 2.
2. Is the unknown simply a function or field satisfying a differential equation plus side conditions?
   - If yes, think direct PDE/BVP solvers or PINNs.
3. Is there an unknown physical parameter hidden in the model?
   - If yes, think inverse methods; PINNs are attractive if data are sparse but the governing equations are trusted.
4. Does the solution contain sharp fronts or multiscale features?
   - If yes, be cautious: vanilla PINNs may need enhancements.

9.9 Notation alignment table

Symbol	Meaning	Where it appears
x	generic state variable or state vector	Notebooks 01, 07
θ, ω	pendulum angle and angular velocity	Notebooks 06, 07
$u(t)$	control input	Notebooks 01, 06, 07
$u(x,t)$	PDE solution field	Notebooks 03, 04, 05
$y(x)$	Bratu solution function	Notebooks 02, 06
J	total objective / cost functional	Notebooks 01, 06, 07
L	running cost, or context-dependent loss notation	several notebooks; always check context
$V(x,t)$	value function	Notebooks 01, 07 benchmark
$P(t)$	Riccati coefficient in quadratic value function	Notebook 01
λ	costate variables	Notebooks 06, 07
r or R	residual of ODE/PDE/HJB equation	most PINN notebooks

Chapter 10 Notation Glossary

Glossary of repeated symbols

- State variables: physical quantities describing the system configuration.
- Control input: decision variable used to influence dynamics.
- Running cost: integrand accumulated over time.
- Terminal cost: penalty applied at the final time.
- Value function: optimal cost-to-go from a state-time pair.
- Hamiltonian: combined object used in HJB and PMP to express trade-offs between cost and dynamics.
- Costate: adjoint variable in PMP.
- Residual: equation mismatch at a collocation point.
- Collocation point: sampled point where the residual is enforced.

Chapter 11 Final Big Picture Conclusion

The tutorial notebooks tell one coherent story. Sometimes the unknown is a solution field; sometimes it is a control input; sometimes it is a value function; sometimes it is both a field and a hidden parameter. The mathematics determines which viewpoint is natural.

For nonlinear BVPs and PDEs, PINNs are attractive because they convert governing equations and side conditions directly into a trainable loss. For inverse problems, PINNs can blend sparse observations with physics in a powerful way. For optimal control, HJB and PMP remain foundational because they express optimality itself, not just approximation.

The most important lesson is not that one method replaces all others. The lesson is that problem structure matters. If students leave the tutorial able to identify the unknown, state the governing equations, classify the conditions as boundary/initial/terminal, and choose a method that matches the mathematical target, then the tutorial has succeeded.

Short Roadmap Recap

- Notebook 01 connected HJB, Riccati theory, and value-function PINNs.
- Notebook 02 showed a clean nonlinear BVP PINN.
- Notebook 03 extended the idea to a space-time PDE.
- Notebook 04 introduced inverse parameter identification.
- Notebook 05 showed a meaningful failure mode.
- Notebook 06 compared methods by mathematical scope.
- Notebook 07 compared trajectory PINNs, DP/HJB ideas, and PMP on nonlinear control.

References

This short reference list collects books and papers that are directly relevant to the mathematics and computational ideas appearing throughout the booklet.

[R1] Richard Bellman, *Dynamic Programming*, Princeton University Press, 1957.

[R2] Dimitri P. Bertsekas, *Dynamic Programming and Optimal Control*, Volumes I and II, Athena Scientific.

[R3] Donald E. Kirk, *Optimal Control Theory: An Introduction*, Dover Publications, 2004.

[R4] H. T. Banks, *Control and Estimation in Distributed Parameter Systems*, SIAM, 1992.

[R5] Lawrence C. Evans, *An Introduction to Mathematical Optimal Control Theory*, lecture notes, University of California, Berkeley.

[R6] Lawrence C. Evans, *Partial Differential Equations*, American Mathematical Society, 2nd edition, 2010.

[R7] J. David Logan, *Applied Mathematics*, Wiley; useful background on ODEs, PDEs, and boundary-value problems.

[R8] Uri M. Ascher, Robert M. M. Mattheij, and Robert D. Russell, *Numerical Solution of Boundary Value Problems for Ordinary Differential Equations*, SIAM, 1995.

[R9] Avner Friedman, *Partial Differential Equations of Parabolic Type*, Dover Publications.

[R10] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, 378 (2019), 686–707.

[R11] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis, "Physics-informed deep learning (Part I): Data-driven solutions of nonlinear partial differential equations," arXiv:1711.10561.

[R12] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis, "Physics-informed deep learning (Part II): Data-driven discovery of nonlinear partial differential equations," arXiv:1711.10566.

[R13] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang, "Physics-informed machine learning," *Nature Reviews Physics*, 3 (2021), 422–440.

[R14] Sifan Wang, Yujun Teng, and Paris Perdikaris, "Understanding and mitigating gradient pathologies in physics-informed neural networks," *SIAM Journal on Scientific Computing*, 43 (2021), A3055–A3081.

[R15] Lu Lu, Xuhui Meng, Zhiping Mao, and George Em Karniadakis, "DeepXDE: A deep learning library for solving differential equations," *SIAM Review*, 63 (2021), 208–228.

[R16] H. J. Kushner and Paul G. Dupuis, *Numerical Methods for Stochastic Control Problems in Continuous Time*, Springer.

[R17] Harold J. Kushner, *Partial Differential Equations and Probability*, relevant background for HJB-type equations.

[R18] E. B. Lee and L. Markus, *Foundations of Optimal Control Theory*, Wiley, 1967.

[R19] Lev S. Pontryagin, V. G. Boltyanskii, R. V. Gamkrelidze, and E. F. Mishchenko, *The Mathematical Theory of Optimal Processes*, Interscience, 1962.

[R20] H. K. Khalil, *Nonlinear Systems*, Prentice Hall; useful for nonlinear dynamics intuition behind the pendulum examples.